Campus Share Engineering Bible v1.0

A practical engineering specification for building a complete college project in one day using React, Vite, Firebase and Tailwind CSS.

1. Project Vision

Campus Share solves a simple problem: students frequently need items such as calculators, lab coats, books, cycles or sports equipment for only a few days. The application allows students to publish listings and contact owners directly. The goal is not to build a commercial platform but to demonstrate authentication, CRUD operations, cloud storage, responsive UI and deployment.

Implementation Notes

Acceptance criteria: the feature should compile without errors, be responsive, use Firebase where required, and remain simple enough for a college viva. Avoid unnecessary libraries or advanced architecture.

2. Scope

Included: Google Login, Browse Listings, Search, Add Listing, Item Details, Contact Owner, Firestore, Firebase Storage and Deployment. Excluded: Payments, Chat, Ratings, Notifications, Admin Dashboard, Maps, Recommendation Engine and AI.

Implementation Notes

Acceptance criteria: the feature should compile without errors, be responsive, use Firebase where required, and remain simple enough for a college viva. Avoid unnecessary libraries or advanced architecture.

3. Tech Stack

Frontend: React + Vite + JavaScript + Tailwind CSS. Backend Services: Firebase Authentication, Firestore, Firebase Storage. Deployment: Firebase Hosting. Version Control: GitHub.

Implementation Notes

Acceptance criteria: the feature should compile without errors, be responsive, use Firebase where required, and remain simple enough for a college viva. Avoid unnecessary libraries or advanced architecture.

4. User Flow

Login → Home → Browse Listings → Search or Open Item → Contact Owner. Authenticated users can also navigate to Add Listing, upload an image, save the listing and see it appear on the home page.

Implementation Notes

Acceptance criteria: the feature should compile without errors, be responsive, use Firebase where required, and remain simple enough for a college viva. Avoid unnecessary libraries or advanced architecture.

5. Folder Structure

src/ components/ pages/ firebase/ services/ hooks/ assets/ App.jsx main.jsx Keep components reusable and pages responsible only for layout.

Implementation Notes

Acceptance criteria: the feature should compile without errors, be responsive, use Firebase where required, and remain simple enough for a college viva. Avoid unnecessary libraries or advanced architecture.

6. Firestore Design

Collection: listings Fields: itemName, description, rentPerDay, category, hostel, contactNumber, imageUrl, ownerUid, createdAt. Each document represents one rentable item.

Implementation Notes

Acceptance criteria: the feature should compile without errors, be responsive, use Firebase where required, and remain simple enough for a college viva. Avoid unnecessary libraries or advanced architecture.

7. Firebase Storage

Store uploaded images inside listing-images/. The Firestore document stores only the download URL returned after upload.

Implementation Notes

Acceptance criteria: the feature should compile without errors, be responsive, use Firebase where required, and remain simple enough for a college viva. Avoid unnecessary libraries or advanced architecture.

8. UI Guidelines

White background, blue primary color, rounded cards, subtle shadows, responsive grid, large CTA buttons and consistent spacing. Tailwind utility classes only.

Implementation Notes

Acceptance criteria: the feature should compile without errors, be responsive, use Firebase where required, and remain simple enough for a college viva. Avoid unnecessary libraries or advanced architecture.

9. Components

Navbar: navigation and logout. SearchBar: realtime filtering. ItemCard: preview card. ListingForm: create listing. ProtectedRoute: block unauthenticated access. LoadingSpinner: loading state.

Implementation Notes

Acceptance criteria: the feature should compile without errors, be responsive, use Firebase where required, and remain simple enough for a college viva. Avoid unnecessary libraries or advanced architecture.

10. Routing

/login /home /add /item/:id Unknown routes redirect to /home after authentication or /login otherwise.

Implementation Notes

Acceptance criteria: the feature should compile without errors, be responsive, use Firebase where required, and remain simple enough for a college viva. Avoid unnecessary libraries or advanced architecture.

11. Coding Standards

Use functional React components, hooks, descriptive names, small reusable components, minimal comments, Tailwind instead of custom CSS unless absolutely required.

Implementation Notes

Acceptance criteria: the feature should compile without errors, be responsive, use Firebase where required, and remain simple enough for a college viva. Avoid unnecessary libraries or advanced architecture.

12. Development Phases

Phase 1: Setup. Phase 2: Authentication. Phase 3: Browse Listings. Phase 4: Add Listing. Phase 5: Search. Phase 6: Item Details. Phase 7: Deployment.

Implementation Notes

Acceptance criteria: the feature should compile without errors, be responsive, use Firebase where required, and remain simple enough for a college viva. Avoid unnecessary libraries or advanced architecture.

13. Testing Checklist

Verify: • Login works • Logout works • Image upload succeeds • Firestore stores listing • Search filters correctly • Mobile layout remains usable • Invalid form submission is blocked

Implementation Notes

Acceptance criteria: the feature should compile without errors, be responsive, use Firebase where required, and remain simple enough for a college viva. Avoid unnecessary libraries or advanced architecture.

14. Viva Notes

Explain: Why React? Component-based UI. Why Firebase? Backend without managing servers. Why Firestore? NoSQL cloud database. What is CRUD? Create, Read, Update and Delete. Why Tailwind? Faster UI development.

Implementation Notes

Acceptance criteria: the feature should compile without errors, be responsive, use Firebase where required, and remain simple enough for a college viva. Avoid unnecessary libraries or advanced architecture.

15. Master AI Prompt

Always read PROJECT.md, ARCHITECTURE.md, TASKS.md and AI_RULES.md before coding. Implement only one phase. Do not invent new features. Explain every created file. Stop after finishing the requested phase.

Implementation Notes

Acceptance criteria: the feature should compile without errors, be responsive, use Firebase where required, and remain simple enough for a college viva. Avoid unnecessary libraries or advanced architecture.